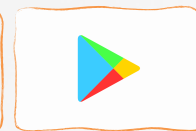


JSON Data

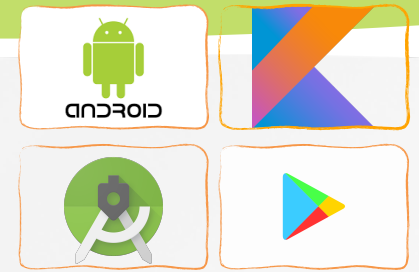
- » **JSON** (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate.
 - It is based on JavaScript.
 - It is language and platform independent.
 - It is easier to read and write than XML.
- » JSON is built on two structures:
 - A collection of name/value pairs. In various languages, this is realized as an *object*, record, struct, dictionary, hash table, keyed list, or associative array
 - An ordered list of values. In most languages, this is realized as an *array*, vector, list, or sequence.



JSON Data

» Nested structures to simply group data together.

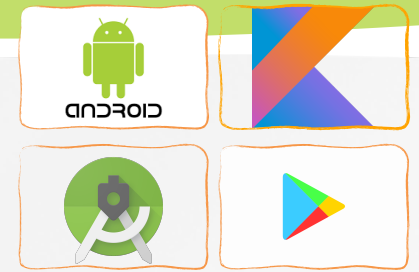
```
{
  "id": "0001",
  "type": "donut",
  "name": "Cake",
  "image":
  {
    "url": "images/0001.jpg",
    "width": 200,
    "height": 200
  },
  "thumbnail":
  {
    "url": "images/thumbnails/0001.jpg",
    "width": 32,
    "height": 32
  }
}
```



Processing JSON Data

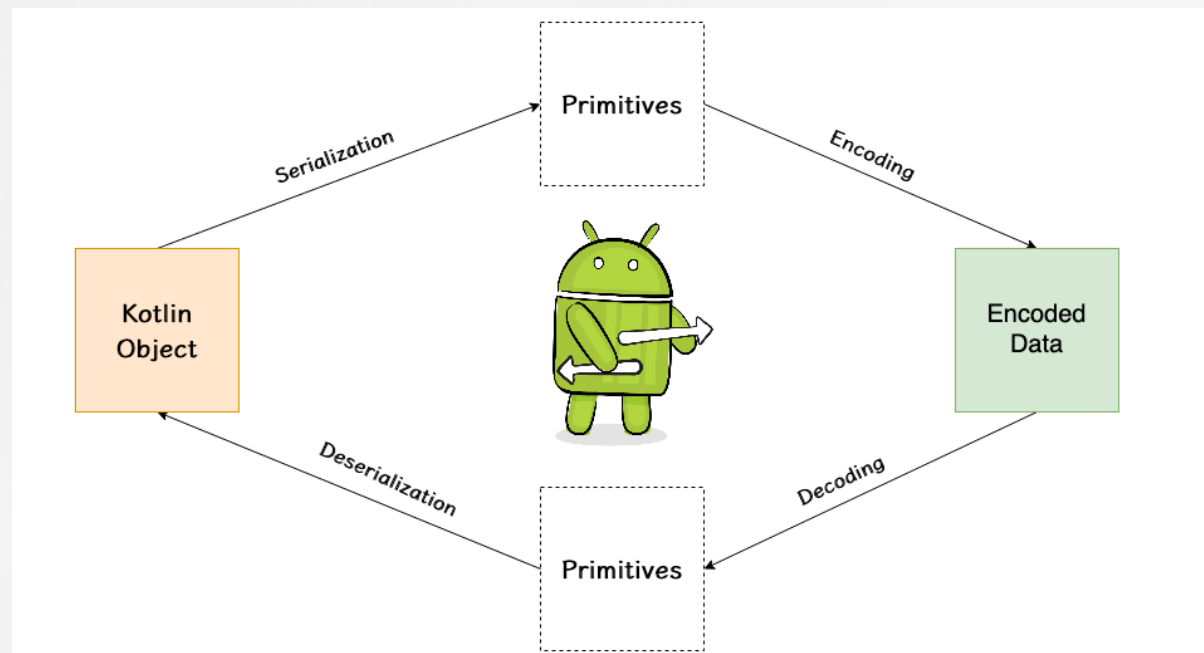
» Understanding Data Decoding and Deserialization

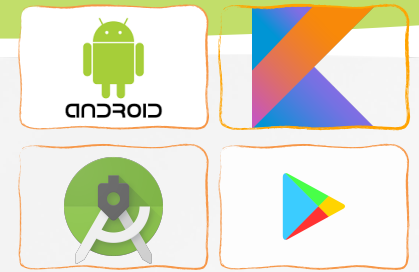
- **Decoding** is the process of parsing encoded data to produce a deserialized representation of an object consisting of its primitive contents while maintaining its structure.
- **Deserialization** is the process of converting such a decoded stream of primitives into an object.



Processing JSON Data

- » The ***Kotlin Serialization library*** (`kotlinx.serialization`) combines a compiler plugin and a runtime API
 - It supports both serialization and deserialization

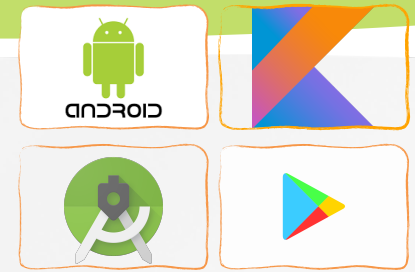




Processing JSON Data

- » Add the following dependencies to your project
 - Open the project level **build.gradle (Project: app)** and add the Kotlin Serialization in the **dependencies** block

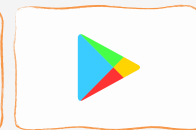
```
1  // Top-level build file where you can add configuration options common to all sub-projects/modules.
2  // Top-level build file where you can add configuration options common to all sub-projects/modules.
3  buildscript {
4      ext.kotlin_version = '1.6.10'
5      dependencies {
6          classpath "org.jetbrains.kotlin:kotlin-serialization:$kotlin_version"
7      }
8  }
9
10 plugins {...}
16
17 task clean(type: Delete) {delete rootProject.buildDir}
```



Processing JSON Data

- Open the project level **build.gradle (Module: app)** and add the *plugins* and **dependencies** block

```
plugins {  
    id 'com.android.application'  
    id 'org.jetbrains.kotlin.android'  
    id 'org.jetbrains.kotlin.plugin.serialization'  
}  
  
dependencies {  
    implementation 'androidx.core:core-ktx:1.7.0'  
    implementation 'androidx.appcompat:appcompat:1.4.1'  
    implementation 'com.google.android.material:material:1.5.0'  
    implementation 'androidx.constraintlayout:constraintlayout:2.1.3'  
    implementation "org.jetbrains.kotlinx:kotlinx-serialization-json:1.3.2"  
}
```



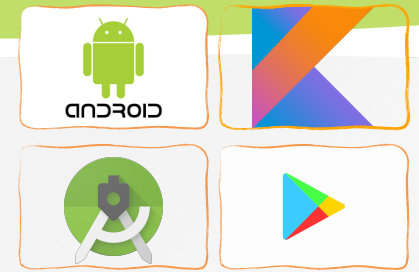
Processing JSON Data

» Data class

```
import kotlinx.serialization.SerialName
import kotlinx.serialization.Serializable

@Serializable
data class CakeInfo(
    @SerialName("id")
    val idCake: String,
    val type: String,
    val name: String,
    val image: PictureInfo,
    val thumbnail: PictureInfo
    val objectNull: Int?
)

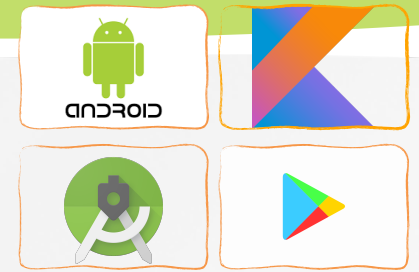
@Serializable
data class PictureInfo(
    val url: String,
    val width: Double,
    val height: Double
)
```



Processing JSON Data

» Reading json data from ***cake.json*** file in ***assets*** folder

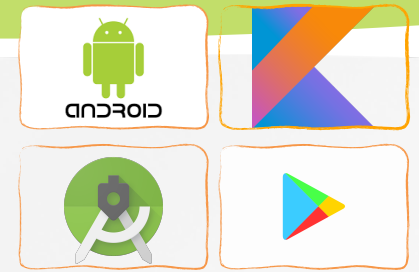
```
fun readJSONFile(){  
    val inputStream = assets.open("cake.json")  
  
    val cake = Json {ignoreUnknownKeys = true; explicitNulls = false }  
        .decodeFromStream<CakeInfo>(inputStream)  
  
    Log.i("hdlog", cake.toString())  
}
```

Processing JSON Data

» Writing your object to json string

```
fun writingObjectToJson(){  
    val image = PictureInfo("image/pizza.jpg", 200.0, 200.0)  
    val thumb = PictureInfo("image/pizza-thumb.jpg", 100.0, 100.0)  
    val cakeInfo = CakeInfo("123", "pizza", "Chicago Pizza",  
        image, thumb, null)  
  
    val format = Json { explicitNulls = false }  
    val jsonString = format.encodeToString(cakeInfo)  
  
    Log.i("hdlog", jsonString)  
}
```



Processing JSON Data

» Code comments:

– `Json {ignoreUnknownKeys = true; explicitNulls = false }`

- skip unknown key if that key has not defined in your data class
- read null value if if that key has not defined in your json file

– Change reading key

```
@SerializedName("id") //key in json ile  
val idCake: String //property in your data class
```